

SimpleImputer#

<https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html>

Description:

Univariate imputer for completing missing values with simple strategies. Replace missing values using a descriptive statistic (e.g. mean, median, or most frequent) along each column, or using a constant value. Read more in the User Guide. Added in version 0.20: SimpleImputer replaces the previous `sklearn.preprocessing.Imputer` estimator which is now removed.

Function Signature

```
class sklearn.impute.SimpleImputer(*, missing_values=nan,
strategy='mean', fill_value=None, copy=True,
add_indicator=False, keep_empty_features=False)[source]#
```

Parameters:

`missing_values`int, float, str, np.nan, None or pandas.NA,
default=np.nan

`strategy`str or Callable, default='mean'

`fill_value`str or numerical value, default=None

```
class sklearn.impute.SimpleImputer(*, missing_values=nan,
strategy='mean', fill_value=None, copy=True,
add_indicator=False, keep_empty_features=False)[source]#
```

Parameters:

`missing_values`int, float, str, np.nan, None or pandas.NA,
default=np.nan

`strategy`str or Callable, default='mean'

`fill_value`str or numerical value, default=None

`fit(X, y=None)[source]#`

Parameters:

`X`{array-like, sparse matrix}, shape (n_samples, n_features)

`y`Ignored

Returns:

Parameters

`X{array}`

Input data, where `n_samples` is the number of samples and `n_features` is the number of features.

`Xarray`

Input samples.

`yarray`

Target values (None for unsupervised transformations).

`**fit_paramsdict`

Additional fit parameters.

`input_featuresarray`

Input features. If `input_features` is `None`, then `feature_names_in_` is used as feature names in. If `feature_names_in_` is not defined, then the following input feature names are generated: `["x0", "x1", ..., "x(n_features_in_ - 1)"]`. If `input_features` is an array-like, then `input_features` must match `feature_names_in_` if `feature_names_in_` is defined.

`paramsdict`

Parameter names mapped to their values.

`Xarray`

The imputed data to be reverted to original data. It has to be an augmented array of imputed data and the missing indicator mask.

`**paramsdict`

Estimator parameters.

`X{array}`

The input data to complete.

Code Examples

```
>>> import numpy as np
>>> from sklearn.impute import SimpleImputer
>>> imp_mean = SimpleImputer(missing_values=np.nan,
strategy='mean')
>>> imp_mean.fit([[7, 2, 3], [4, np.nan, 6], [10, 5, 9]])
SimpleImputer()
>>> X = [[np.nan, 2, 3], [4, np.nan, 6], [10, np.nan, 9]]
>>> print(imp_mean.transform(X))
[[ 7.    2.    3. ]
 [ 4.    3.5   6. ]
 [10.    3.5   9. ]]
```

Notes & Warnings

NOTE See also `IterativeImputerMultivariate` imputer that estimates values to impute for each feature with missing values from all the others. `KNNImputerMultivariate` imputer that estimates missing features using nearest samples.

Detailed Documentation

Documentation

Univariate imputer for completing missing values with simple strategies.

Replace missing values using a descriptive statistic (e.g. mean, median, or most frequent) along each column, or using a constant value.

Read more in the User Guide.

Added in version 0.20: `SimpleImputer` replaces the previous `sklearn.preprocessing.Imputer` estimator which is now removed.

The placeholder for the missing values. All occurrences of `missing_values` will be imputed. For pandas' dataframes with nullable integer dtypes with missing values, `missing_values` can be set to either `np.nan` or `pd.NA`.

The imputation strategy.

If “mean”, then replace missing values using the mean along each column. Can only be used with numeric data.

If “median”, then replace missing values using the median along each column. Can only be used with numeric data.

If “most_frequent”, then replace missing using the most frequent value along each column. Can be used with strings or numeric data. If there is more than one such value, only the smallest is returned.

If “constant”, then replace missing values with `fill_value`. Can be used with strings or numeric data.

If an instance of `Callable`, then replace missing values using the scalar statistic returned by running the callable over a dense 1d array containing non-missing values of each column.

Added in version 0.20: `strategy="constant"` for fixed value imputation.

Added in version 1.5: `strategy=callable` for custom value imputation.

When `strategy == "constant"`, `fill_value` is used to replace all occurrences of `missing_values`. For string or object data types, `fill_value` must be a string. If `None`, `fill_value` will be 0 when imputing numerical data and `"missing_value"` for strings or object data types.

If `True`, a copy of `X` will be created. If `False`, imputation will be done in-place whenever possible. Note that, in the following cases, a new copy will always be made, even if `copy=False`:

If `X` is not an array of floating values;

If `X` is encoded as a CSR matrix;

If `add_indicator=True`.

If `True`, a `MissingIndicator` transform will stack onto output of the imputer's transform. This allows a predictive estimator to account for missingness despite imputation. If a feature has no missing values at fit/train time, the feature won't appear on the missing indicator even if there are missing values at transform/test time.

If `True`, features that consist exclusively of missing values when `fit` is called are returned in results when `transform` is called. The imputed value is always 0 except when `strategy="constant"` in which case `fill_value` will be used instead.

Added in version 1.2.

Changed in version 1.6: Currently, when `keep_empty_feature=False` and `strategy="constant"`, empty features are not dropped. This behaviour will change in version 1.8. Set `keep_empty_feature=True` to preserve this behaviour.

The imputation fill value for each feature. Computing statistics can result in np.nan values. During transform, features corresponding to np.nan statistics will be discarded.

Indicator used to add binary indicators for missing values. None if add_indicator=False.

Number of features seen during fit.

Added in version 0.24.

Names of features seen during fit. Defined only when X has feature names that are all strings.

Added in version 1.0.

Multivariate imputer that estimates values to impute for each feature with missing values from all the others.

Multivariate imputer that estimates missing features using nearest samples.

Columns which only contained missing values at fit are discarded upon transform if strategy is not "constant".

In a prediction context, simple imputation usually performs poorly when associated with a weak learner. However, with a powerful learner, it can lead to as good or better performance than complex imputation such as IterativeImputer or KNNImputer.

For a more detailed example see [Imputing missing values before building an estimator](#).

Fit the imputer on X.

Input data, where n_samples is the number of samples and n_features is the number of features.

Not used, present here for API consistency by convention.

Fit to data, then transform it.

Fits transformer to `X` and `y` with optional parameters `fit_params` and returns a transformed version of `X`.

Target values (None for unsupervised transformations).

Additional fit parameters.

Get output feature names for transformation.

If `input_features` is `None`, then `feature_names_in_` is used as feature names in. If `feature_names_in_` is not defined, then the following input feature names are generated: `["x0", "x1", ..., "x(n_features_in_ - 1)"]`.

If `input_features` is an array-like, then `input_features` must match `feature_names_in_` if `feature_names_in_` is defined.

Transformed feature names.

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

A `MetadataRequest` encapsulating routing information.

Get parameters for this estimator.

If `True`, will return the parameters for this estimator and contained subobjects that are estimators.

Parameter names mapped to their values.

Convert the data back to the original representation.

Inverts the transform operation performed on an array. This operation can only be performed after SimpleImputer is instantiated with `add_indicator=True`.

Note that `inverse_transform` can only invert the transform in features that have binary indicators for missing values. If a feature has no missing values at fit time, the feature won't have a binary indicator, and the imputation done at transform time won't be inverted.

Added in version 0.24.

The imputed data to be reverted to original data. It has to be an augmented array of imputed data and the missing indicator mask.

The original X with missing values as it was prior to imputation.

Set output container.

See [Introducing the set_output API](#) for an example on how to use the API.

Configure output of transform and fit_transform.

"default": Default output format of a transformer

"pandas": DataFrame output

"polars": Polars output

None: Transform configuration is unchanged

Added in version 1.4: "polars" option was added.

Set the parameters of this estimator.

The method works on simple estimators as well as on nested objects (such as Pipeline). The latter have parameters of the form `<component>__<parameter>` so that it's possible to update each component of a nested object.

Estimator parameters.

Impute all missing values in X.

The input data to complete.

X with imputed values.

Column Transformer with Mixed Types

Combine predictors using stacking

Imputing missing values with variants of IterativeImputer

Imputing missing values before building an estimator

Permutation Importance vs Random Forest Feature Importance (MDI)

Displaying estimators and complex pipelines

Introducing the `set_output` API

Release Highlights for scikit-learn 0.23

Release Highlights for scikit-learn 1.1

Release Highlights for scikit-learn 1.5

© Copyright 2007 - 2025, scikit-learn developers (BSD License).